Workflow

How a change gets from your editor to production, and the commands that do it.

The two commands

Command	Branch	Use for
`git fix "message"`	`bug-fix`	Minor changes, corrections, anything small
`git feat "message"`	`features`	New functionality

```
git fix  "category names are case-insensitive"
git feat "csv export for the register"
```

Each one, in order: fetches, switches to the branch (creating it **from `main`** if it doesn't exist), stages everything with `git add -A`, commits with your message, rebases on anything the pipeline pushed back down, then pushes.

That push is the trigger. There is nothing else to run.

Never commit to `main` by hand. The pipeline fast-forwards `main`, and a manual commit there makes that fast-forward fail.

What happens after you push

`.github/workflows/promote.yml` wakes up and:

1. Runs `npm run build`. **Red build → everything stops here**, nothing moves.
2. Checks whether the changeset touches `migrations/`.
3. Green and no migrations → merges into `upgrade`, fast-forwards `main`, pushes.
4. Back-merges `main` into `bug-fix` and `features` so neither drifts.

Vercel sees `main` move and deploys. Typical end-to-end: about two minutes.

Watching it

```
gh run list --limit 5              # recent runs and their outcome
gh run watch                       # follow the run in progress
gh run view --log-failed           # why the last one failed
```

Or the **Actions** tab on GitHub.

When it stops on purpose

Two things hold a change back. Both open a pull request instead of failing, and neither loses your work — the branch keeps everything you pushed.

1. The changeset touches `migrations/`

Migrations are run **by hand** in the Supabase SQL editor. Code that reaches production ahead of its schema is broken code, so the pipeline refuses to guess.

```
push → build ✓ → migration detected → PR opened into upgrade → STOP
```

To finish it:

1. Open the PR (`gh pr list`). Its body names the `.sql` files.
2. Run those files in the Supabase SQL editor, in numerical order.
3. Merge the PR.
4. `upgrade.yml` builds and takes it to `main`.

Merging the PR *is* the statement "the schema is already in place". Nothing else checks, so don't merge before you've run the SQL.

While a migration sits unmerged, every further push to that branch is held too — the migration is still in the diff against `main`. This is a feature: keep working and pushing, and it all accumulates into the same PR. Nothing escapes to production until you run the SQL and merge.

2. The branch conflicts with `upgrade`

Usually because `bug-fix` and `features` touched the same lines. Resolve the conflict in the PR and merge it.

Recipes

Ship a small fix

```
git fix "round loan interest to 2dp"
```

Ship a feature

```
git feat "export register as csv"
```

Ship a change that includes a migration

```
git fix "case-insensitive categories"     # held; opens a PR
gh pr list                                # find its number
# ...run the .sql files in the Supabase SQL editor...
gh pr merge <number> --merge              # promotion resumes
```

Check where everything is

```
git ls-remote origin main upgrade bug-fix features
```

Retry a promotion without a new commit — after fixing something on GitHub's side, or to re-run a red build:

```
gh run rerun <run-id>
gh workflow run promote.yml --ref bug-fix
```

Push `upgrade` to production by hand — if `upgrade` is green and you want it out now:

```
gh workflow run upgrade.yml --ref upgrade
```

Troubleshooting

"Everything up-to-date", no run started. Nothing was committed — the working tree was already clean. Check `git status`.

Nothing happened after the push. You were on the wrong branch. Only `bug-fix`, `features`, and `upgrade` trigger workflows. `git rev-parse --abbrev-ref HEAD`.

The run failed at "Fast-forward main". Something was committed to `main` directly, so `main` is no longer an ancestor of `upgrade`. Fix by merging `main` into `upgrade` and pushing `upgrade`:

```
git switch upgrade && git pull && git merge origin/main && git push
```

The run failed at "Build". The same failure reproduces locally:

```
npm ci && npm run build
```

Note it builds without `.env` — missing Supabase keys produce a "Not configured" screen at runtime, not a build error, so a green CI build says nothing about whether Vercel's env vars are set.

A back-merge warning in the log. `main` couldn't be merged back into a working branch cleanly. Nothing is broken; resolve it locally:

```
git switch features && git pull
```

Push rejected as non-fast-forward. The pipeline moved the branch under you. The aliases handle this themselves; if you pushed by hand, `git pull --rebase`.

Setup on a fresh clone

The aliases are local git config, so they don't travel with the repo:

```
git config --local alias.fix  '!f() { b=bug-fix;  git fetch -q origin; git
switch "$b" 2>/dev/null || git switch -c "$b" origin/main || return 1; git
add -A || return 1; git commit -m "$1" || return 1; if git ls-remote --
exit-code --heads origin "$b" >/dev/null 2>&1; then git pull --rebase --
autostash origin "$b" || return 1; fi; git push -u origin "$b"; }; f'
git config --local alias.feat '!f() { b=features; git fetch -q origin; git
switch "$b" 2>/dev/null || git switch -c "$b" origin/main || return 1; git
add -A || return 1; git commit -m "$1" || return 1; if git ls-remote --
exit-code --heads origin "$b" >/dev/null 2>&1; then git pull --rebase --
autostash origin "$b" || return 1; fi; git push -u origin "$b"; }; f'
```

Repo settings this depends on (already applied):

- **Settings → Actions → General → Workflow permissions**: *Read and write*, and allow Actions to create pull requests. Without it the pipeline can't push or open the migration PR.
- **Leave `main` unprotected.** Required reviews reject the bot's push and deadlock every promotion.
- **Vercel**: production branch is `main`. `upgrade`, `bug-fix` and `features` get preview deploys — those are your staging URLs.

What this does and doesn't protect you from

The only automatic gate is `npm run build`. It catches a broken import, a syntax error, a missing file. It does **not** catch a wrong number in `src/lib/analytics.js` — there are no tests in this project. Anything you want reviewed, review before you push.

What it does protect: your data. The frontend is a static build and nothing in this pipeline can reach Postgres. Schema changes only ever happen when you run SQL yourself, which is exactly what the migration hold is

there to enforce. See Pushing updates.